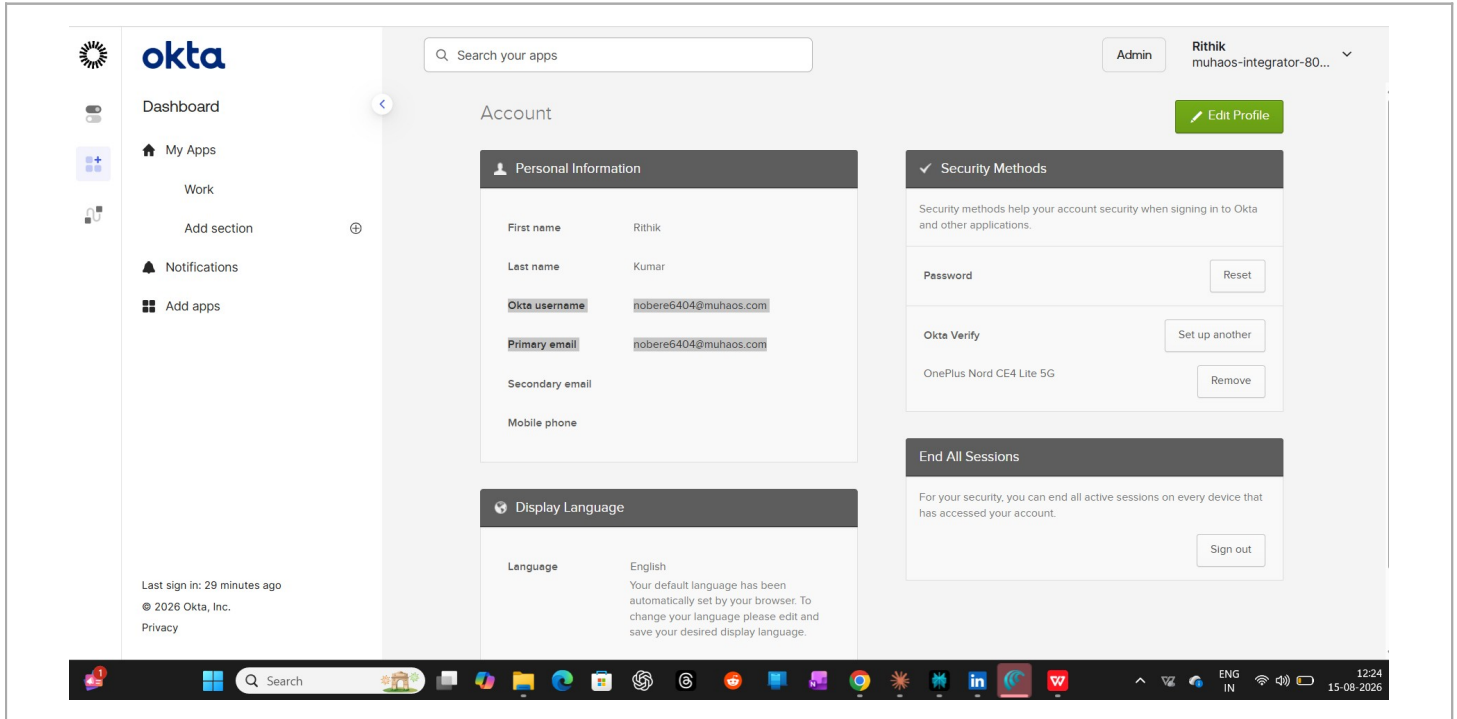


End-to-End JML Workflow in Okta

A hands-on Joiner → Mover → Leaver simulation, documented by Rithik Kumar

I built this project to practice the exact identity lifecycle I've studied through SailPoint IdentityIQ and Okta — onboarding an employee, moving them across departments, and offboarding them — in a real Okta environment I set up myself, rather than just reading about it.

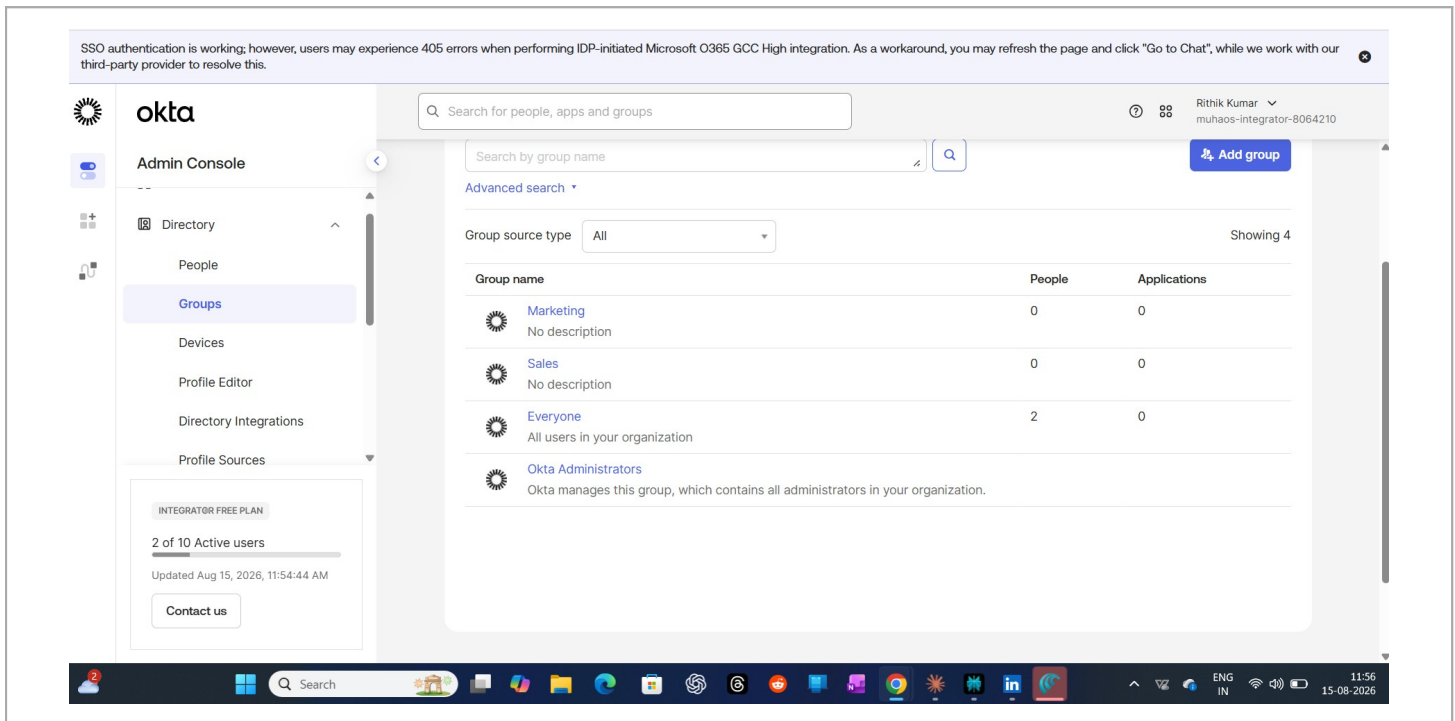
Below is my Okta account/tenant page, showing the developer org I created specifically for this practice.



Note: This is a dummy Okta Developer tenant I created for this practice project. It's valid for 30 days and not tied to any employer or production system. Okta username / primary email used: nobere6404@muhaos.com

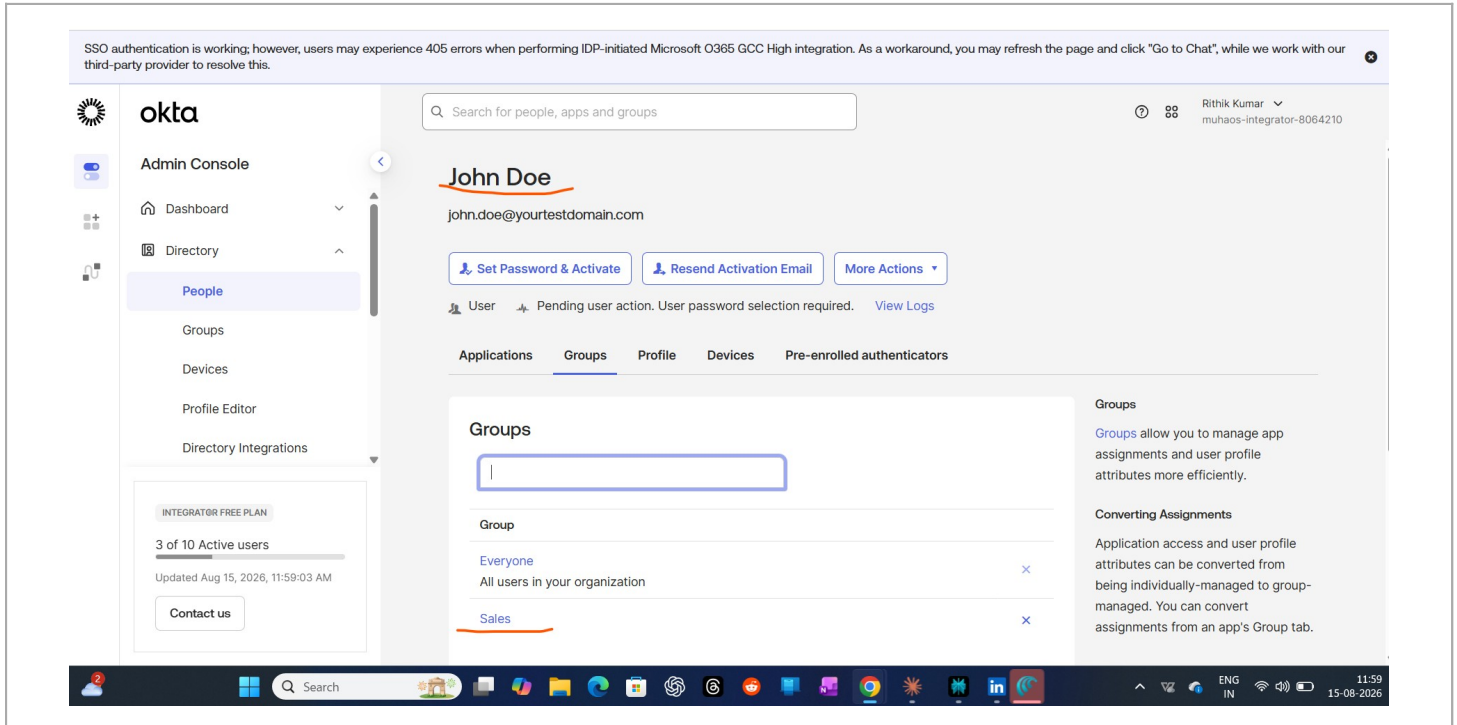
Step 1 — Groups Created (Joiner setup)

First thing I did was set up Sales and Marketing as security groups. The plan was to manage access at the group level instead of assigning things to each person individually — that way, moving someone between departments later would be a lot cleaner.



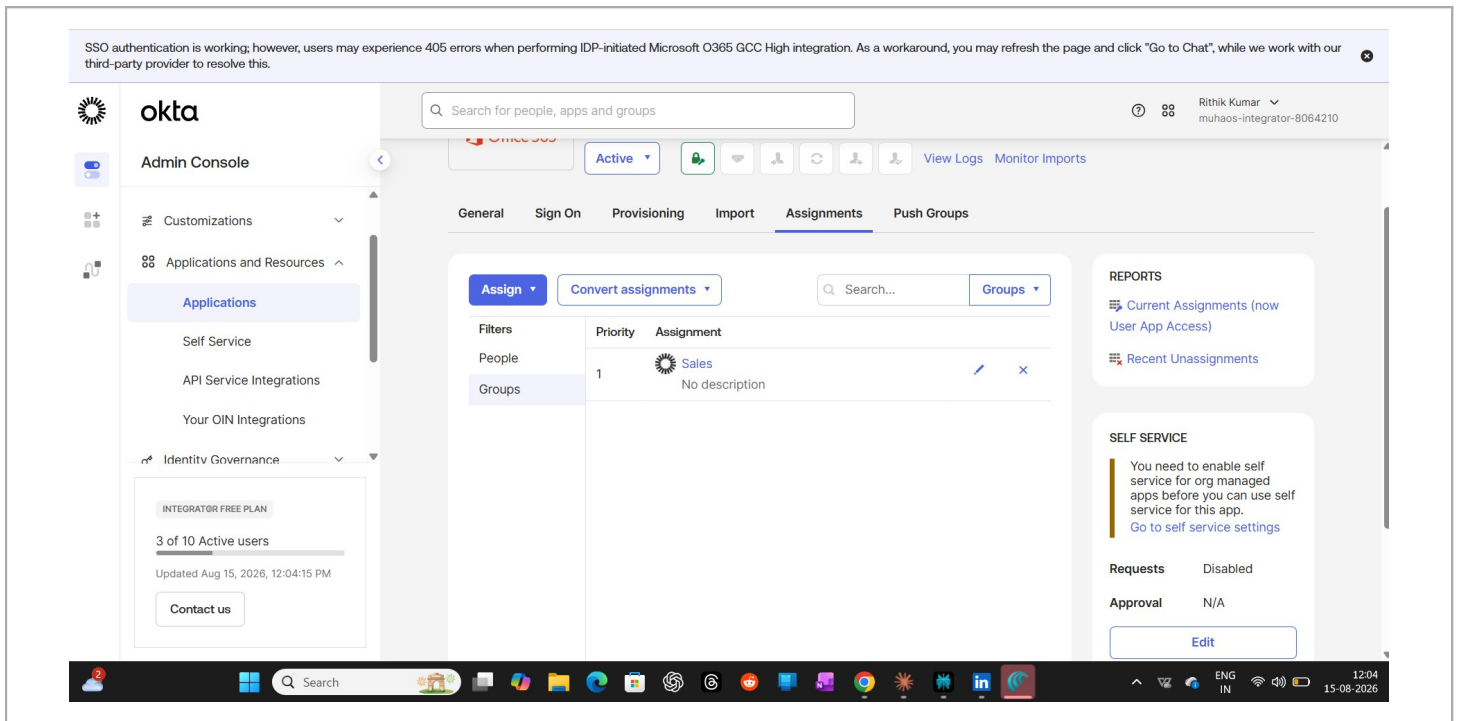
Step 2 — New Employee Onboarded

Created a test user, John Doe, and added him to the Sales group right away. Since access is tied to group membership, he inherits whatever Sales has as soon as he's added — no separate app assignment needed for him individually.



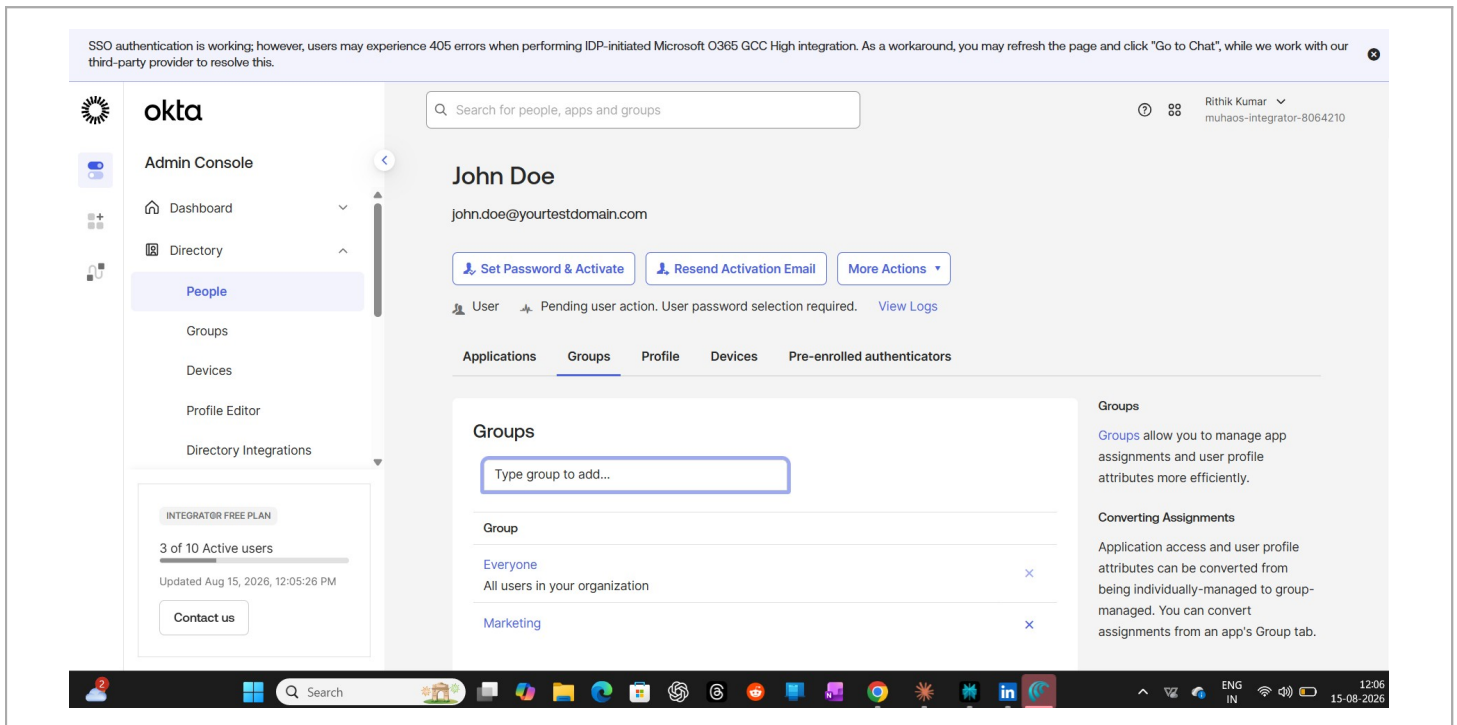
Step 3 — Office 365 Access Granted via Group

Assigned the Office 365 app to the Sales group (not to John individually). This is the part that matters for real environments — group-based assignment means access scales automatically as people join or leave the group, instead of an admin manually touching every user's app list.



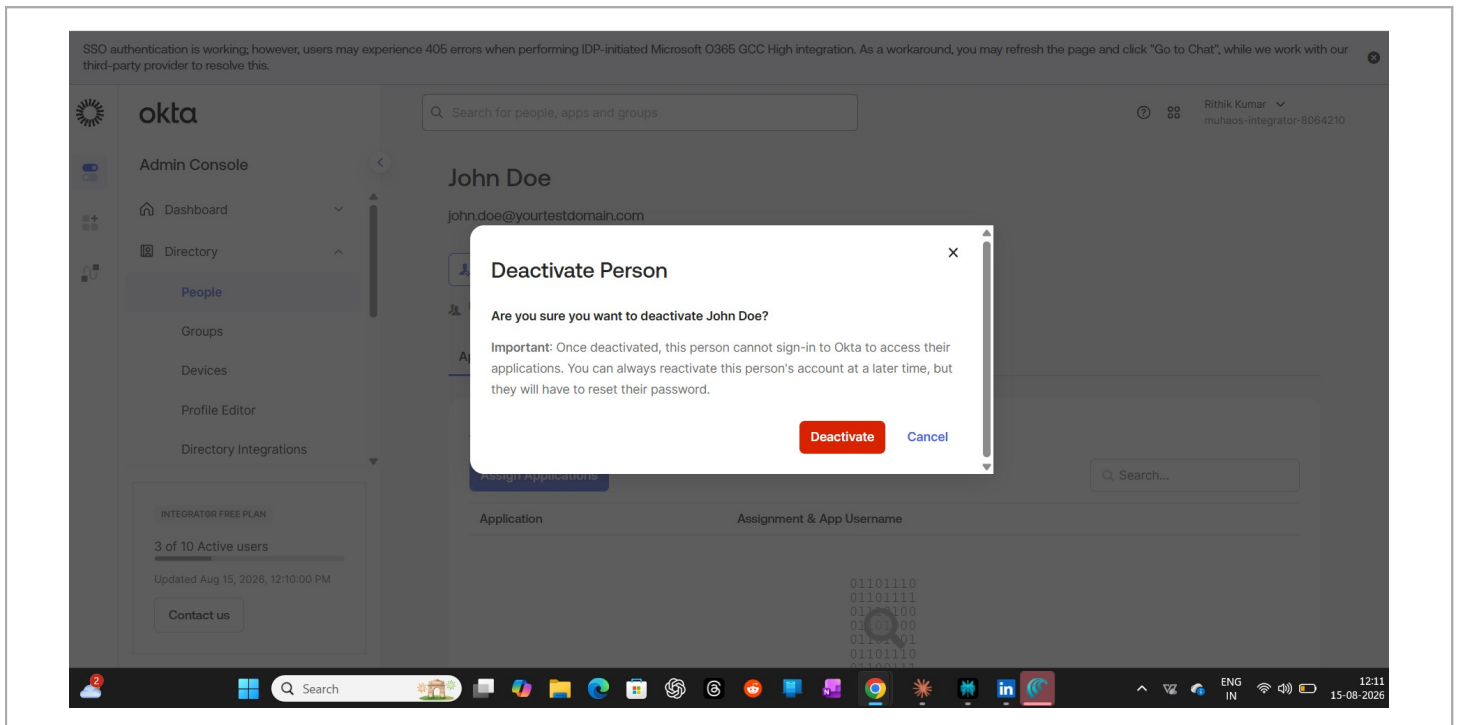
Step 4 — Mover — Moved from Sales to Marketing

Simulated a department transfer: removed John from Sales and added him to Marketing. Because his old access was tied to the Sales group and not assigned to him directly, it dropped automatically the moment he left that group — nothing left over from his old role.



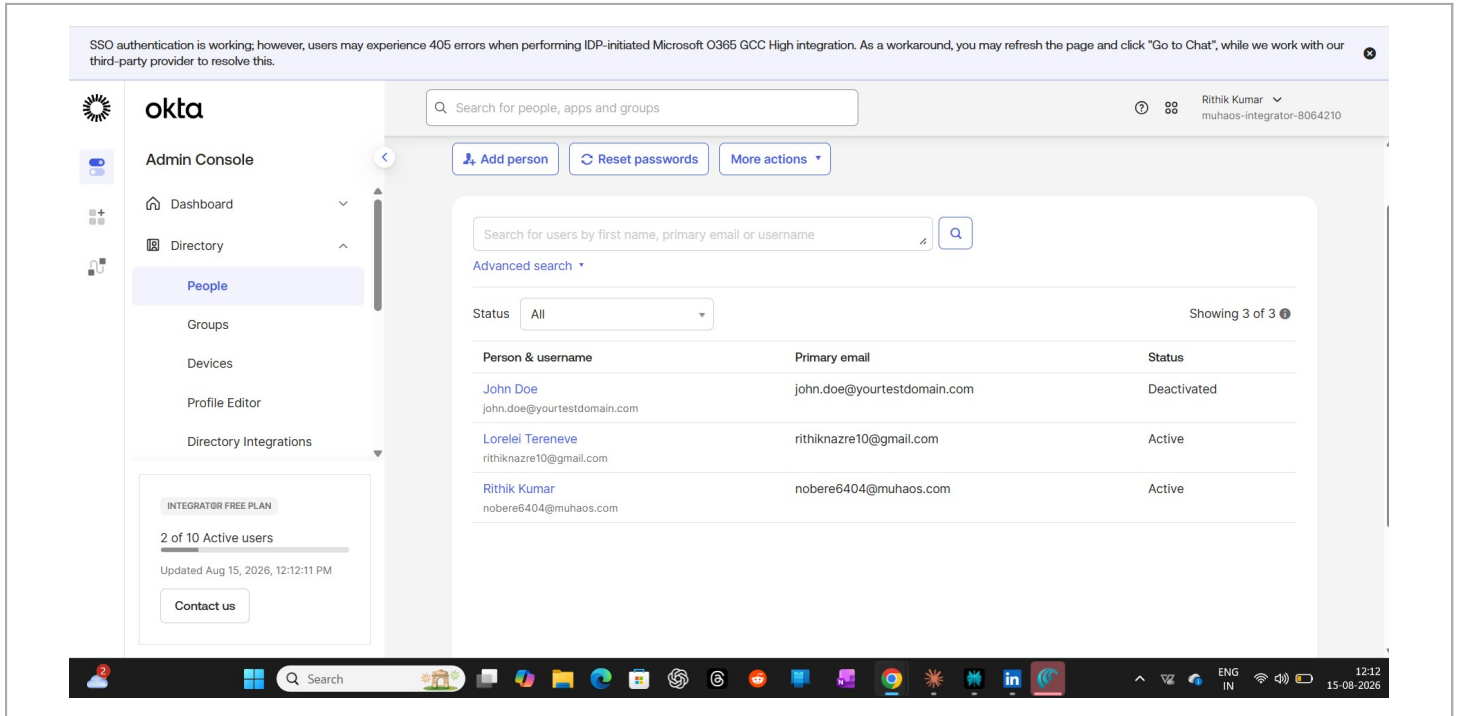
Step 5 — Leaver — Offboarding Triggered

For offboarding, I used Okta's Deactivate action instead of manually removing groups and apps one by one. This is the single-action approach real IAM teams rely on for leavers — it immediately revokes all app access and kills any active sessions at once, so there's no gap where a terminated employee could still be logged in somewhere.



Step 6 — Leaver — Deactivation Confirmed

Final check: John Doe now shows as Deactivated in the directory. His identity record is retained (for audit/history purposes, same as production systems do), but he has zero active access anywhere.



What I took away from this

Doing this hands-on made the Mover and Leaver steps click in a way reading about them didn't. Group-based access means the Mover step is basically free — you're not hunting down every app the person had. And the Leaver step showed me why a single deactivation action matters so much: it removes the chance of an admin forgetting to revoke one app out of five during manual offboarding.